Bellissemo Pizza Forecasting System

Project Final Report

Kelvin Asu Ekuri
David Daniel
Danil Afonchikov
Dadakhon Turgunboev

1 Introduction and project Description

The food service industry faces significant operational challenges due to unpredictable customer demand, particularly in fast-casual restaurant chains where timing and inventory management are critical to profitability. Traditional approaches rely heavily on manual interpretation of historical sales data, often lacking the precision required for effective short-term operational planning.

This project addresses these challenges by developing an AI-powered forecasting system for Bellissimo Pizza, a growing restaurant chain in Uzbekistan. The system leverages historical transaction data, weather information, holiday patterns, and departmental characteristics to generate precise predictions for hourly revenue and order volumes across 24, 48, and 72-hour horizons.

The solution employs modern data engineering principles, utilizing Apache Airflow for workflow orchestration, PostgreSQL for data storage, MinIO for model management, and Streamlit for user interaction. Key innovations include cluster-based modeling for computational efficiency, hierarchical prediction models, and comprehensive integration of external factors for enhanced accuracy.

1.1 Problem Statement

Bellissemo Pizza, a growing fast-casual restaurant chain in Uzbekistan, faces critical operational challenges stemming from unpredictable customer demand. The business must frequently make time-sensitive decisions regarding:

- Inventory replenishment
- Staff scheduling
- Kitchen preparation
- Promotion planning

However, the absence of an integrated, predictive analytics system often leads to overstocking, understaffing, and missed promotional opportunities. Current practices rely heavily

on manual interpretation of historical data, without the accuracy or granularity needed to optimize short-term operations.

Given that Bellissemo Pizza already maintains digital records (e.g., POS data, promotion schedules), an opportunity exists to integrate these sources into a machine learning-based forecasting system that enhances decision-making and operational efficiency.

1.2 Project Vision

This project proposes the development of an AI-powered time-series forecasting system tailored for Bellissemo Pizza. The system will:

- Predict hourly revenue, order volumes, and item-level demand (e.g., pizza type, size, toppings)
- Generate forecasts over 24h, 48h, and 72h horizons
- Enable data-driven decisions across operations, procurement, and marketing
- Integrate with Bellissemo Pizza's existing IT infrastructure (databases, reporting tools)

The solution will evolve from a minimum viable product (MVP) to a more robust, scalable platform that includes automated retraining, drift monitoring, and business alerts.

2 Project Objectives

The system is expected to:

1. Ingest and process data from various sources, including sales, weather and holiday calendars.
2. Generate accurate forecasts at multiple time horizons.
3. Provide a user-friendly interface to visualize predictions and key metrics.
4. Automatically retrain forecasting models on a scheduled (e.g., weekly) basis.
5. Integrate with existing internal tools and workflows.

3 System Scope and Features

3.1 Phase 1 – MVP Development (Weeks 1–4)

- Data profiling
- ETL pipeline with Apache Airflow
- Different models for forecasting order volumes and revenue (lightgbm, xgboost, random forest and ridge)
- PostgreSQL models registry and predictions storage
- MinIO storage for models and intermediate files with versioning
- Dashboard using Streamlit
- CI/CD integration via GitHub

3.2 Phase 2 – Updates and bug fixes (Weeks 5–8)

- Dashboard updates
- Model versioning
- User roles and management
- Weekly model validation after training
- Critical bug fixes

4 Functional Requirements

FR ID	Functional Requirement
FR1	The system should ingest structured data from internal (e.g., POS, CRM) and external (e.g., weather APIs) sources.
FR2	The system should process raw inputs into time-series features such as lagged values, rolling means, and event indicators.
FR3	The system should forecast hourly revenue and order volume with a target business accuracy (4.1) in the range of 90–95%, based on validation data.
FR4	The system should include a user interface (UI) for interacting with forecasts, visualizing trends, and accessing logs.
FR5	The system should have user management and access control.
FR6	The system should have model and data versioning.
FR7	The system should retrain forecasting models weekly using newly accumulated data.
FR9	The system should allow integration or export of forecast data into existing reporting tools.

4.1 Business Accuracy Metric

The business accuracy metric is calculated as follows:

Let P = Predicted value and A = Actual value.

$$\text{Accuracy} = \begin{cases} 100, & \text{if } P = A = 0 \\ 0, & \text{if } A = 0 \text{ and } P \neq 0 \\ \max\left(0, \frac{A - |P - A|}{A}\right) \times 100, & \text{if } A > 0 \end{cases}$$

$$\text{Overall Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \text{Accuracy}(P_i, A_i)$$

5 Non-Functional Requirements

NFR ID	Non-Functional Requirement
NFR1	The user interface should be intuitive and accessible to non-technical users.
NFR2	The system should be compatible with existing IT infrastructure (e.g., PostgreSQL).
NFR3	System components should support modular deployment and allow updates without requiring system-wide reconfiguration.
NFR4	Model accuracy metrics (e.g., MAPE, F1-score) should be visible in the dashboard for review and comparison.

6 Design Constraints

DC ID	Design Constraint
DC1	The system should use open-source tools, e.g., Prophet, XGBoost, TFT, Airflow, etc.
DC2	Data and forecasts should be stored in PostgreSQL or a similar system already used by the client.
DC3	The architecture should be containerized for portability (e.g., via Docker).
DC4	The system should include logging for data pipeline and model events.

7 Stakeholder Requirements

SR ID	Stakeholder	Requirement
SR1	Operations Team	Should receive hourly forecasts to support shift planning and kitchen prep.
SR2	Executives	Should monitor key metrics.
SR3	IT / Data Analysts	Should be able to connect forecast data to existing business intelligence tools.

8 Integration Considerations

The system needs to be designed to integrate with Bellissimo Pizza's existing IT systems, including:

- PostgreSQL databases
- CSV/JSON export for BI/reporting tools such as Excel or Power BI

9 Architecture

The Bellissimo AI forecasting system is organized into five logical layers: data sources, processing pipeline, ML engine storage, prediction services, and user interface. Each layer is modular and serves a distinct role—from ingesting and transforming data, to training models, generating forecasts, and delivering results to users. This architecture enables scalable, maintainable, and accurate demand forecasting across all departments.

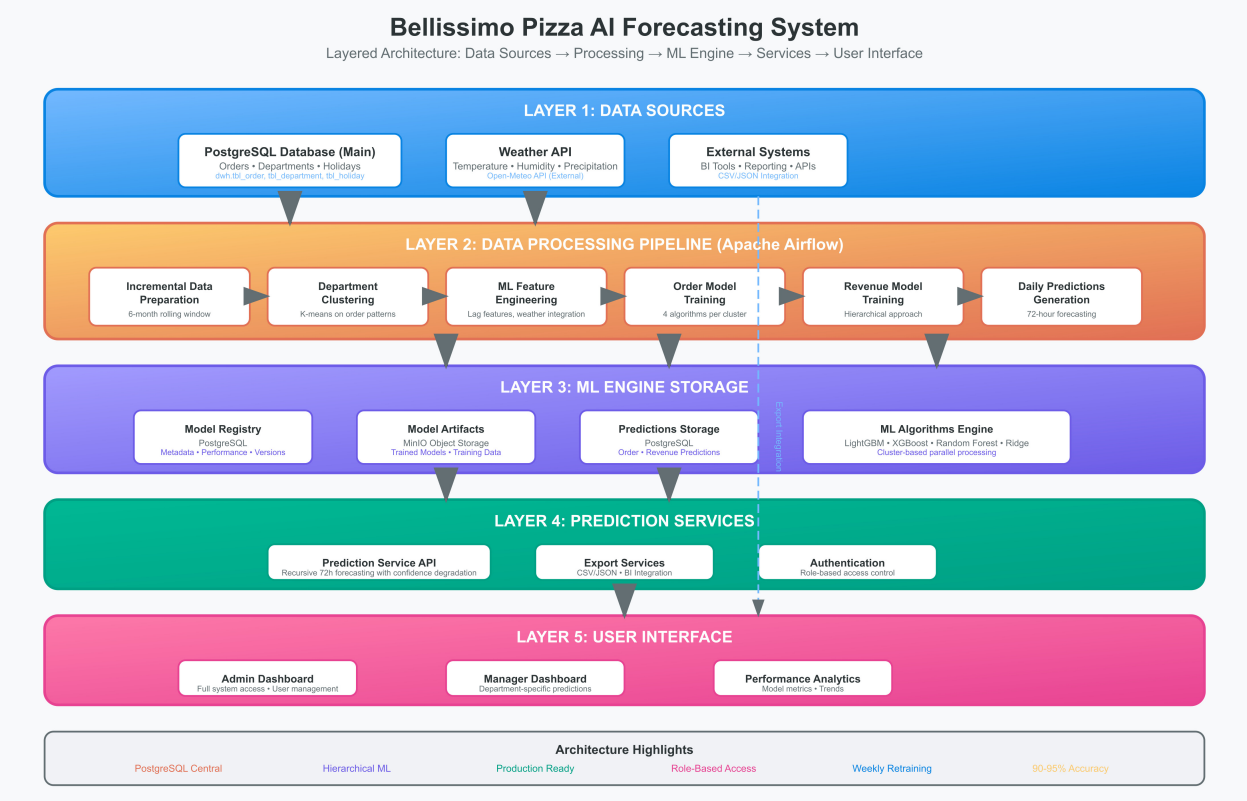


Figure 1: Layered Architecture of the Bellissimo Pizza AI Forecasting System

10 Dataset overview and Data Description

The Bellissimo Pizza forecasting system operates on a dataset integrating multiple data sources to capture factors influencing restaurant demand. The data architecture consists of four primary components: transactional data, departmental information, external environmental factors, and derived analytical features.

10.1 Primary Data Sources

Transaction Data (dwh.tbl_order) serves as the core dataset, containing historical order transactions with the following attributes:

- `order_id`: Unique identifier for each transaction
- `department_id`: Restaurant location identifier
- `order_type`: Classification of order method (delivery, pickup, dine-in)
- `client_phone`: Customer identifier for repeat customer analysis
- `order_status`: Transaction status ('done' for completed orders)
- `order_created_datetime`: Precise timestamp of order placement
- `order_date`: Date of order creation
- `order_done_datetime`: Timestamp when order was completed
- `order_items_cnt`: Number of items in the order
- `price_full`: Full price before discounts
- `price_vat`: VAT amount
- `price_discount`: Discount amount applied
- `price_net_revenue`: Final revenue after discounts and taxes
- `platform`: Order origination platform
- `coupon_name`: Applied coupon identifier

Order Detail Data (dwh.tbl_order_dish) provides granular item-level information with attributes:

- `order_id`: Foreign key linking to main order table
- `dish_price_net_revenue`: Revenue contribution per item
- `dish_item_cnt`: Quantity of specific items ordered

Department Information (dwh.tbl_department) contains restaurant location characteristics:

- `department_id`: Unique location identifier
- `department_name`: Human-readable location name
- `city`: Geographic location for weather correlation
- `type`: Restaurant format classification
- `is_franchise`: Franchise vs corporate indicator
- `latitude`: Geographic coordinate for weather API
- `longitude`: Geographic coordinate for weather API
- `is_active`: Operational status filter

Holiday Calendar Data (dwh.tbl_holiday) provides temporal context with attributes:

- `st_date`: Date of holiday or special event
- `holiday_name`: Description of holiday or event
- `official`: Government-recognized holiday indicator
- `weekend`: Weekend classification
- `ramadan_fasting`: Religious observance indicator
- `rest_day`: Rest day indicator
- `school_vacation`: School calendar impact factor
- `short_day`: Abbreviated business day indicator

10.2 Feature Engineering Pipeline

The system implements feature engineering at hourly granularity, creating multiple categories of predictive features:

Temporal Features capture cyclical patterns through hour, dayofweek, month, quarter, and binary indicators for weekends and peak hours (11–13, 18–20). Cyclical transformations using sine and cosine functions ensure proper handling of circular time features.

Lag Features capture temporal dependencies across multiple periods: 1h, 6h, 12h, 24h, 48h, and 168h (weekly) for both order counts and revenue values.

Rolling Statistics include moving window statistics (means and standard deviations) over 6h, 12h, 24h, and 168h periods, plus exponential moving averages with different decay parameters.

Weather Features integrate real-time data from Open-Meteo API including temperature, humidity, precipitation, wind_speed, weather_code, cloud_cover, surface_pressure, and derived categorical features for weather conditions and temperature perception.

Business Context Features incorporate department cluster assignments, encoded department types, franchise indicators, and department-specific peak hour identification.

Holiday and Event Features include public holiday indicators, official weekend designations, Ramadan periods, and school vacations.

10.3 Data Quality and Preprocessing

The system implements data quality measures including duplicate removal, filtering for completed orders, revenue validation against and active department filtering.

Transaction-level data is aggregated to hourly intervals, calculating order counts, total revenue, average revenue, total items, and delivery counts per hour. This aggregation forms the foundation for forecasting while maintaining the granularity necessary for operational decision-making.

11 Models Used

In this project, we trained four machine learning models: LightGBM, XGBoost, Random Forest, and Ridge Regression. Each model brings distinct characteristics to the forecasting task:

LightGBM is a gradient boosting framework that uses a histogram-based approach and leaf-wise tree growth. Developed by Microsoft, it offers fast training speeds and efficient handling of large datasets, which can be advantageous for time-series forecasting tasks like restaurant demand prediction.

XGBoost is another gradient boosting implementation known for its regularization capabilities. Its depth-wise tree construction and built-in cross-validation features help capture complex patterns while mitigating overfitting risks.

Random Forest employs an ensemble approach through bootstrap sampling and multiple decision trees. This method can reduce prediction variance and provide stable forecasts, which may be particularly useful for noisy transactional data.

Ridge Regression is a linear model with L2 regularization that can efficiently handle high-dimensional feature spaces. Its straightforward architecture offers computational advantages and provides interpretable coefficients.

The system implements a hierarchical modeling approach where order count predictions are generated first using one of the models, then these predictions serve as additional features for revenue forecasting models, creating a two-stage prediction pipeline that recognizes the fundamental dependency between order volume and revenue.

Model selection is performed based on quantitative evaluation metrics, allowing the system to choose the most appropriate algorithm for the task at hand.

11.1 Model Versioning

The system implements a semantic versioning scheme for model management using the format `v{major}.{minor}`. All models initially start at version `v1.0`.

The versioning logic follows these rules:

- **Minor increment**: When the same algorithm is retrained with new data (e.g., `v1.0` → `v1.1`). This occurs during scheduled retraining cycles where the model architecture remains unchanged but parameters are updated with fresh data.
- **Major increment**: When the algorithm itself is changed (e.g., from Ridge Regression to Random Forest), the major version increases and minor resets to zero (e.g., `v1.1` → `v2.0`). This indicates a fundamental change in the modeling approach.

Each model version is stored in the registry tables (`order_model_registry` and `revenue_model_registry`) with metadata including training date, performance metrics, and feature sets. The versioned model artifacts are persisted in MinIO storage, enabling the system to track model evolution, compare performance across versions, and rollback to previous versions if needed. This approach ensures model traceability and supports systematic model improvement over time.

12 Difficulties Encountered

During the development of this project, we've encountered several difficulties:

- **Granularity of predictions**

  The client's requirements included highly detailed forecasts, such as predicting not only order volumes but also the specific dish IDs within each order (e.g., "order1": [581, 624], "order2": [969, 3049, ...]). While this would have provided highly detailed outputs, such fine-grained prediction lies beyond the capabilities of standard demand forecasting approaches, which are designed to model aggregate trends rather than individual item sequences. After evaluating the technical feasibility and resource requirements, this aspect was deprioritized in favor of a more practical, high-level forecasting approach.

- **Delayed and evolving data access**

Initial data was provided in .csv format, requiring a complete pipeline redesign when database access was later granted. Critical data for item-level predictions (e.g., pizza types, quantities, and groupings) arrived too late in the project timeline, preventing even a baseline solution for this subtask.

- **Scalability constraints in modeling**

  A single model approach proved insufficient due to poor prediction quality across different departments. Conversely, training separate models per department was computationally infeasible, leading to a trade-off between accuracy and scalability. The resulting cluster-based approach successfully reconciled prediction quality with system performance.